

Reinforcement Learning–Based Artillery Targeting System

Manas Mondal (Team Lead)
Arkaprava Jas (Team Member 2)
CS 246: Artificial Intelligence

December 17, 2025

Abstract

This project investigates the application of reinforcement learning techniques to an artillery targeting problem under uncertain environmental conditions. A custom Gymnasium-based environment is developed to simulate projectile dynamics influenced by gravity and wind. Two reinforcement learning approaches are implemented and compared: tabular Q-learning and the Soft Actor-Critic (SAC) algorithm. Experimental results demonstrate that while Q-learning performs well in discretized settings, SAC provides superior performance, stability, and generalization in continuous action spaces.

1 Introduction

Artillery targeting under environmental uncertainty is a challenging control problem involving delayed rewards and stochastic dynamics. Wind disturbances make analytical solutions unreliable, motivating the use of learning-based approaches. Reinforcement Learning (RL) enables an agent to learn optimal firing strategies through repeated interaction with the environment.

The objectives of this project are:

- To model artillery targeting as a Markov Decision Process (MDP).
- To design a custom Gymnasium-compatible environment.
- To implement and compare tabular Q-learning and Soft Actor-Critic (SAC).
- To evaluate performance based on accuracy and efficiency.

2 Problem Formulation and Representation

2.1 Problem Domain

The problem domain consists of a stationary artillery cannon that must hit a fixed target placed at a random horizontal distance. Projectile motion is affected by gravity and wind, introducing uncertainty. The agent must select firing parameters that maximize the probability of hitting the target with minimal shots.

2.2 State, Actions, and Constraints

2.2.1 State Representation

The state representation specifies all information available to the agent at a given time step that is required to make optimal decisions. In this project, the state is represented as a continuous-valued vector:

$$s = [\theta, x_t, y_t, w] \tag{1}$$

where:

- θ is the current gun elevation angle (in degrees).
- x_t is the horizontal position of the target.
- y_t is the vertical position of the target (fixed at ground level).
- w denotes the wind speed affecting projectile motion.

This state representation satisfies the Markov property, as it contains all necessary information to predict future dynamics without requiring past states.

Assumptions from the implementation:

- The cannon position is fixed throughout all episodes.
 - Wind remains constant within an episode but varies across episodes.
 - The target is stationary and does not move during an episode.
 - Air resistance other than wind is neglected.
 - Gravity is constant and uniform.
-

2.2.2 Action Representation

The action space defines the control variables available to the agent. Actions are represented as a continuous vector:

$$a = [\Delta\theta, v] \tag{2}$$

where:

- $\Delta\theta$ represents the incremental change applied to the gun angle.
- v denotes the muzzle velocity of the projectile.

For the Q-learning agent, the continuous action space is discretized into a finite set of angle adjustments and velocity values. In contrast, the SAC agent operates directly on the continuous action space, enabling smoother and more precise control.

Assumptions from the implementation:

- Angle adjustments are applied incrementally rather than resetting the gun angle.
 - The firing action is instantaneous, and projectile motion is simulated after action execution.
 - Only one projectile is active at a time.
-

2.2.3 Constraints and Rules

To ensure physical realism and stable learning, the following constraints are imposed:

- **Angle Constraint:**

$$0^\circ \leq \theta \leq 90^\circ \quad (3)$$

- **Velocity Constraint:** Muzzle velocity is bounded within predefined minimum and maximum limits.
- **Shot Limit Constraint:** Each episode allows a fixed maximum number of shots.
- **Wind Constraint:** Wind values are bounded within a fixed range.

Assumptions from the implementation:

- Projectile collision is approximated using a distance threshold around the target.
 - Terrain is flat, and obstacles are not present.
 - Episodes terminate immediately upon a successful hit.
-

2.2.4 Goal Test

An episode is considered successfully solved when the projectile lands within a predefined radius of the target. Upon satisfying this condition, the episode terminates and a success reward is assigned.

2.2.5 Rationale

This formulation balances realism and computational tractability. The chosen state captures essential physical and environmental variables, continuous actions allow precise control, and constraints prevent physically invalid behavior. The assumptions simplify the environment while preserving the core learning challenges of uncertainty and control.

3 Data / Environment

3.1 Reinforcement Learning Environment

A custom environment is implemented using Gymnasium. Each episode begins with a random target location and wind value.

Observation Space: A continuous Box space representing gun angle, target position, and wind.

Action Space: A continuous Box space defining angle change and firing speed.

Reward Structure:

- Large positive reward for a successful hit
- Negative reward proportional to distance from target otherwise
- Penalty for multiple shots

Termination Conditions: An episode ends upon a hit or after a maximum number of shots.

4 Methodology

4.1 Projectile Motion Model

The environment dynamics are governed by classical projectile motion equations under gravity, with the inclusion of wind as a horizontal disturbance. These equations define the transition model $T(s, a)$ of the Markov Decision Process.

Let θ denote the firing angle and v the muzzle velocity selected by the agent. The initial velocity components are given by:

$$v_x = v \cos(\theta), \quad v_y = v \sin(\theta) \quad (4)$$

Wind is modeled as an additive horizontal velocity component w , resulting in the effective horizontal velocity:

$$v_x^{\text{eff}} = v_x + w \quad (5)$$

Assuming constant gravitational acceleration g , the projectile position at time t is computed as:

$$x(t) = x_0 + v_x^{\text{eff}} t \quad (6)$$

$$y(t) = y_0 + v_y t - \frac{1}{2}gt^2 \quad (7)$$

where (x_0, y_0) is the fixed cannon position.

The projectile trajectory is simulated until it intersects the ground. A hit is registered when the Euclidean distance between the projectile landing position (x_f, y_f) and the target position (x_t, y_t) satisfies:

$$\sqrt{(x_f - x_t)^2 + (y_f - y_t)^2} \leq \varepsilon \quad (8)$$

where ε is a predefined hit tolerance radius.

Model Assumptions

- Gravitational acceleration g is constant.
- Air resistance is neglected except for the wind term.
- Wind remains constant during each episode.
- Projectile motion is deterministic given the state and action.

This physics-based model provides a deterministic and interpretable transition function while maintaining sufficient complexity to challenge reinforcement learning algorithms under uncertainty.

4.2 Algorithms Used

4.2.1 Tabular Q-Learning

Q-learning is a model-free, value-based reinforcement learning algorithm that learns an optimal action-value function $Q(s, a)$. The agent improves its estimates using observed rewards and transitions without requiring a model of the environment. Since Q-learning operates on discrete spaces, all continuous state and action variables in this project are discretized into finite bins.

Q-learning is employed as a baseline method due to its simplicity, interpretability, and effectiveness for small-scale reinforcement learning problems.

4.2.2 Soft Actor-Critic (SAC)

Soft Actor-Critic is an off-policy, actor-critic reinforcement learning algorithm specifically designed for continuous control tasks. SAC simultaneously learns a stochastic policy (actor) and two action-value functions (critics). In addition to maximizing expected cumulative reward, SAC maximizes policy entropy, which encourages exploration and leads to stable learning behavior.

SAC is particularly well-suited for the artillery targeting task because accurate firing requires smooth and precise continuous actions.

4.3 Justification of Algorithms

Q-learning was selected to demonstrate classical reinforcement learning principles and to provide a transparent performance baseline. However, discretization significantly limits its scalability. SAC overcomes these limitations by operating directly in continuous state and action spaces, offering better generalization and robustness to environmental uncertainty such as wind.

4.4 Mathematical Formulation

The artillery targeting problem is formulated as a Markov Decision Process (MDP):

$$\mathcal{M} = (S, A, T, R, \gamma) \quad (9)$$

where:

- S denotes the continuous state space consisting of gun angle, target position, and wind.
- A denotes the continuous action space defined by angle adjustment and muzzle velocity.
- T represents the transition dynamics governed by projectile motion under gravity and wind.
- R is the reward function.
- $\gamma \in (0, 1)$ is the discount factor.

Reward Function The reward function is designed to encourage accurate and efficient target hits while penalizing poor shots and excessive attempts. The base reward is defined as:

$$R_{\text{base}} = \begin{cases} +R_{\text{hit}}, & \text{if the target is hit} \\ -d, & \text{otherwise} \end{cases} \quad (10)$$

where d denotes the Euclidean distance between the projectile landing position and the target.

To improve learning efficiency, additional reward shaping terms are applied during SAC training. The final reward used by the SAC agent is:

$$R(s, a, s') = R_{\text{base}} + \lambda_1 \frac{1}{1+d} - \lambda_2 N_{\text{shots}} - \lambda_3 |w| \quad (11)$$

where:

- λ_1 rewards proximity to the target even when a hit is not achieved,
- N_{shots} penalizes multiple firing attempts within an episode,
- λ_3 penalizes large wind magnitudes to encourage compensation,
- w is the wind speed.

This shaped reward provides dense feedback and strongly encourages one-shot solutions.

Q-Learning Update Rule For tabular Q-learning, the action-value function is updated as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (12)$$

Soft Actor-Critic Objective The Soft Actor-Critic algorithm optimizes a stochastic policy by maximizing the expected cumulative reward along with an entropy regularization term:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \pi} \left[\sum_t R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right] \quad (13)$$

where $\mathcal{H}(\pi(\cdot | s_t))$ denotes the entropy of the policy, encouraging exploration and stable learning.

5 Implementation

5.1 Environment Design

A custom Gymnasium-compatible environment was implemented to simulate the artillery firing scenario. Each episode initializes a random target position and wind condition. Projectile motion follows standard kinematic equations under gravity, with wind affecting horizontal velocity. A hit is registered when the projectile lands within a predefined distance threshold from the target.

5.2 State and Action Representation

The state vector is defined as:

$$s = [\theta, x_t, y_t, w] \quad (14)$$

where θ is the gun angle, (x_t, y_t) is the target position, and w represents wind speed.

The action vector is given by:

$$a = [\Delta\theta, v] \quad (15)$$

where $\Delta\theta$ is the angle adjustment and v is the muzzle velocity.

5.3 Reward Design and Shaping

The base reward assigns a large positive value for a successful hit and a negative distance-based penalty for missed shots. Reward shaping techniques are applied to accelerate learning, including bonuses for proximity to the target, additional rewards for one-shot hits, and penalties for multiple shots and strong wind conditions.

5.4 Q-Learning Implementation

Continuous state and action variables are discretized into bins for gun angle, distance to target, wind, and action choices. A four-dimensional Q-table stores action-value estimates. An ε -greedy exploration strategy with gradual decay is used during training, and a greedy policy is applied during evaluation.

5.5 SAC Implementation

The SAC agent is implemented using the Stable-Baselines3 framework. Neural networks parameterize the policy and value functions. Observation normalization and reward shaping wrappers are applied to improve training stability. Training is conducted for several hundred thousand timesteps using an off-policy replay buffer on a CPU-based system.

5.6 Visualization and Evaluation

A PyGame-based interface visualizes the artillery system, projectile trajectories, and target. Agent performance is evaluated using hit rate, one-shot success rate, and average number of shots per episode.

6 Results

The performance of the reinforcement learning agents was evaluated through systematic testing in the custom artillery environment. The primary objective of the evaluation was to assess accuracy, efficiency, and robustness of the learned policies under varying wind conditions and target placements.

6.1 Evaluation Metrics

The performance of the reinforcement learning agents was evaluated using the following quantitative metrics.

- **Hit Rate:** This metric measures how reliably the agent is able to hit the target across multiple episodes.

$$\text{Hit Rate} = \frac{N_{\text{hits}}}{N_{\text{episodes}}} \times 100 \quad (16)$$

where N_{hits} denotes the number of episodes in which the target was successfully hit, and N_{episodes} is the total number of evaluation episodes.

- **One-Shot Success Rate:** This metric measures the percentage of episodes in which the agent hits the target using exactly one shot, reflecting optimal and confident decision-making.

$$\text{One-Shot Success Rate} = \frac{N_{\text{one-shot hits}}}{N_{\text{episodes}}} \times 100 \quad (17)$$

where $N_{\text{one-shot hits}}$ represents the number of episodes solved in a single shot.

- **Average Shots per Episode:** This metric evaluates firing efficiency by measuring how many shots the agent requires on average before episode termination.

$$\text{Average Shots} = \frac{1}{N_{\text{episodes}}} \sum_{i=1}^{N_{\text{episodes}}} S_i \quad (18)$$

where S_i is the number of shots taken in the i^{th} episode.

6.2 Q-Learning Results

The Q-learning agent exhibited gradual improvement throughout training. Due to discretization of both state and action spaces, learning required a large number of training episodes to converge.

After convergence, the agent achieved a high overall hit rate. Reward shaping played a crucial role in improving performance, particularly by encouraging one-shot solutions through additional incentives. Many episodes were solved within one or two shots, indicating that the agent successfully learned an approximate mapping between target distance, wind conditions, and firing parameters.

However, the Q-learning agent showed sensitivity to discretization boundaries. Performance degraded slightly under wind values not well represented during training, highlighting limited generalization capability. Additionally, the size of the Q-table increased rapidly with finer discretization, making the approach memory-intensive and less scalable.

6.3 Soft Actor-Critic (SAC) Results

The SAC agent demonstrated significantly faster convergence compared to Q-learning. Entropy regularization enabled stable exploration, preventing premature convergence and oscillatory behavior during training.

The trained SAC policy achieved a very high hit rate and consistently produced one-shot solutions. Because SAC operates directly in continuous action space, it generated smooth and precise angle and velocity adjustments. This allowed the agent to generalize effectively to unseen wind conditions and maintain strong performance across a wide range of scenarios.

Overall, SAC exhibited superior robustness, stability, and adaptability compared to the tabular approach.

6.4 Comparative Analysis

Table 1 presents a comparative summary of Q-learning and SAC performance.

Table 1: Performance Comparison Between Q-Learning and SAC

Metric	Q-Learning	SAC
Hit Rate	High	Very High
One-Shot Success Rate	Moderate-High	Very High
Convergence Speed	Slow	Fast
Stability	Moderate	High
Scalability	Low	High

6.5 Visualization-Based Evaluation

Qualitative evaluation using the PyGame visualization further supported the quantitative results. Q-learning trajectories showed coarse adjustments caused by discretized action selection, whereas SAC-generated trajectories were smooth and physically realistic. The SAC agent consistently achieved accurate first-shot hits, visually confirming its superior control capability.

Overall, the results demonstrate that continuous-control reinforcement learning algorithms such as SAC are better suited for artillery targeting problems involving environmental uncertainty.

7 Discussion

- Both Q-learning and Soft Actor-Critic (SAC) successfully learned firing strategies for the artillery targeting problem, validating the reinforcement learning formulation.
- The Q-learning agent demonstrated that classical tabular reinforcement learning can solve the problem when the state and action spaces are discretized appropriately.
- However, Q-learning required extensive training and was highly sensitive to discretization choices and reward shaping.
- Reward shaping played a critical role in improving learning efficiency by providing dense feedback and encouraging one-shot solutions.
- The SAC agent achieved faster convergence, higher stability, and better generalization due to its continuous action representation and entropy-based exploration.
- SAC-generated firing trajectories were smooth and precise, whereas Q-learning trajectories were comparatively coarse because of discretized action selection.

- Visualization using the PyGame interface helped qualitatively validate the learned policies and confirmed quantitative performance metrics.
- Key limitations observed include high training time, sensitivity to reward design, and simplified environmental assumptions such as a stationary target and flat terrain.

8 Conclusion

- This project successfully modeled artillery targeting as a Markov Decision Process and solved it using reinforcement learning techniques.
- A custom Gymnasium environment was developed to simulate projectile dynamics under wind uncertainty.
- Tabular Q-learning served as an effective baseline but showed limitations in scalability and generalization due to discretization.
- Soft Actor-Critic (SAC) outperformed Q-learning by achieving faster convergence, higher hit rates, and consistent one-shot accuracy.
- Continuous-control reinforcement learning methods were shown to be more suitable for physics-based targeting problems.
- The project objectives of accuracy, efficiency, and robustness were largely achieved.
- Future extensions include moving targets, complex terrain, stochastic wind patterns, and multi-agent coordination.

9 References

References

- [1] Sutton, R. S., and Barto, A. G., *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [2] Haarnoja et al., “Soft Actor-Critic Algorithms and Applications,” 2018.
- [3] Gymnasium Documentation. <https://gymnasium.farama.org>
- [4] Stable-Baselines3 Documentation. <https://stable-baselines3.readthedocs.io>

10 Appendix

Additional screenshots, reward curves, and core code snippets are provided separately as part of the project submission.